

Topic 3: Enumerated Data Type (enum)

1. High-Yield Conceptual Theory

1.1 Definition, Semantics & Memory Representation

An enumeration (`enum`) is a user-defined data type in C consisting of a set of named integer constants called **enumerators**.

```
enum TagName {  
    CONST1,  
    CONST2,  
    CONST3  
} variable_list;
```

Key Architectural Rules

- Underlying Storage Type:** Under the C standard (C89/C99/C11), the underlying integer type for an `enum` is implementation-defined, but it must be an integer type capable of representing all enumerator values (commonly `int`, making `sizeof(enum TagName) == sizeof(int) = 4 bytes`).
- Type of an Enumerator Constant:** In C, individual enumerator constants (e.g., `CONST1`) have the type `int` (unlike C++ where they are strongly typed to the `enum`). Consequently, `sizeof(CONST1) == sizeof(int)`.
- R-value Nature:** Enumerator identifiers are compile-time constants (rvalues). You **cannot** assign values to them at runtime, nor can you apply the address-of operator `&`:

```
CONST1 = 10; // COMPILE ERROR: lvalue required  
int *p = &CONST1; // COMPILE ERROR: cannot take address of rvalue constant
```

1.2 Enumeration Value Assignment & Progression Mechanics

The values assigned to enumerators follow strict sequential rules:

- Default Rule:** The first enumerator defaults to integer index `0`. Each subsequent enumerator increments by `+1` from the preceding enumerator.
- Explicit Assignment Rule:** Any enumerator can be explicitly initialized to any integer constant expression (positive, negative, or zero).

- **Reset Progression:** If an enumerator is assigned explicitly, subsequent unassigned enumerators continue incrementing by `+1` from that new value.

Structural Demonstration

```
enum State {
    A,           // Default: 0
    B = 10,      // Explicit: 10
    C,           // Auto-increment from B: 11
    D = -3,      // Explicit: -3
    E,           // Auto-increment from D: -2
    F = 10,      // Duplicate value allowed: 10
    G            // Auto-increment from F: 11
};
```

- **Duplicates Allowed:** Multiple enumerators can have identical numeric values (`B == F == 10` and `C == G == 11`).

1.3 Scope, Namespace, and Identifier Resolution

1. **Namespace Collision:** In C, enumerator constants share the **ordinary identifier namespace** with:
 - Local and global variables
 - Function names
 - `typedef` names

Unlike `struct` or `union` member names (which have their own member namespaces), enum constants **cannot** share names with other identifiers in the same scope:

```
int x = 5;
enum Test { x, y }; // COMPILE ERROR: redefinition of 'x'
```

2. **Block Scope & Shadowing:** An inner block can declare a variable that shadows an outer enum constant:

```
enum { VAL = 100 };
void func() {
    int VAL = 20; // Legal: local variable 'VAL' shadows the enum constant
    printf("%d", VAL); // Prints 20
}
```

1.4 Comparative Analysis: enum vs. #define vs. const int

Characteristic	enum	#define	const int
Processed Phase	Compiler	Preprocessor	Compiler
Type Checking	Promoted to int	None (raw token text)	Strict type checking
Scope Enforced	Follows C block scope	Global (until #undef)	Follows C block scope
Symbol Table Presence	Yes (available in debuggers)	No (stripped before compiling)	Yes
Addressable (&)	No (pure rvalue)	No (literal token)	Yes (occupies memory if referenced)
switch-case Valid Label	Yes (constant expression)	Yes (if resolves to int)	No (not a constant expression in C)

GATE Insight: In standard C (unlike C++), a `const int x = 5;` is a **read-only variable**, not a true constant expression. Therefore, `case x:` in a `switch` statement causes a compilation error in C, whereas an `enum` constant is a compile-time constant and is completely valid in `case` labels.

2. Common GATE Pitfalls & Edge Cases

1. Duplicate Case Values in switch via Enums

When using enumerators with identical values in a `switch` statement:

```
enum Colors { RED = 1, BLUE = 2, GREEN = 1 };
...
switch (val) {
    case RED:    ... break;
    case GREEN: ... break; // COMPILE ERROR: duplicate case value '1'
}
```

- **Explanation:** Even though `RED` and `GREEN` have different names, the compiler resolves them to their numeric values. Duplicate `case` constants are forbidden in C.

2. Assignment of Arbitrary Integers to enum Variables in C

In standard C, enumeration variables do not enforce strict type safety:

```
enum Direction { NORTH, SOUTH, EAST, WEST };
enum Direction d = 999; // Perfectly VALID in C (promoted to int)
```

- The compiler treats `enum Direction` as an integer type. Any integer value can be stored in `d`, even if it does not match any enumerator.

3. Enumerator Expressions Derived from Preceding Enumerators

An enumerator definition can use previous enumerator values in its constant expression:

```
enum Flags {  
    READ = 1,  
    WRITE = READ << 1,      // 2  
    EXEC = READ << 2,       // 4  
    ALL = READ | WRITE | EXEC // 7  
};
```

- This is valid because all expressions are evaluated at compile time.

3. Previous Year GATE Questions with Step-by-Step Solutions

PYQ 1 (GATE CS 2004 — Conceptual Core)

Question:

Consider the following C declaration:

```
enum day {  
    Mon = 1,  
    Tue,  
    Wed,  
    Thu,  
    Fri,  
    Sat,  
    Sun  
};  
enum day d;
```

If `d = Wed;`, what will be the value printed by `printf("%d", d + 3);`?

Step-by-step Solution:

1. Determine the integer value of each enumerator:

- `Mon = 1`
- `Tue = 1 + 1 = 2`

- `Wed = 2 + 1 = 3`
 - `Thu = 4, Fri = 5, Sat = 6, Sun = 7.`
2. The assignment `d = Wed;` assigns the value `3` to `d`.
 3. The expression `d + 3` evaluates as integer addition:

$$3 + 3 = 6$$

4. `%d` prints the integer `6` (which numerically corresponds to `Sat`).

Output: `6`

PYQ 2 (Adapted from GATE CS / C Standards)

Question:

Consider the following C program:

```
#include <stdio.h>
enum test {
    X,
    Y = 5,
    Z,
    W = Z + 4
};

int main() {
    printf("%d %d %d %d", X, Y, Z, W);
    return 0;
}
```

What is the printed output?

Step-by-step Solution:

1. Trace the progression of each enumerator constant:
 - `X`: First element, unassigned $\rightarrow X = 0$.
 - `Y`: Explicitly initialized to `5` $\rightarrow Y = 5$.
 - `Z`: Unassigned, follows `Y` $\rightarrow Z = Y + 1 = 5 + 1 = 6$.
 - `W`: Explicit constant expression `+ 4` $\rightarrow W = Z + 4 = 6 + 4 = 10$.
2. The `printf` format specifier `%d` outputs each value separated by spaces.

Output: `0 5 6 10`

4. Tricky Practice Questions (Edge Cases & Traps)

Practice Question 1: Negative Values & Arithmetic Progression

Question:

What will be the output of the following C program?

```
#include <stdio.h>
int main() {
    enum Numbers {
        A = -2,
        B,
        C,
        D = 10,
        E
    };
    printf("%d, %d, %d, %d, %d", A, B, C, D, E);
    return 0;
}
```

Detailed Explanation:

1. Trace step-by-step:

- $A = -2$
- $B = A + 1 = -2 + 1 = -1$
- $C = B + 1 = -1 + 1 = 0$
- $D = 10$ (Explicitly set)
- $E = D + 1 = 10 + 1 = 11$

2. Printing yields: $-2, -1, 0, 10, 11$.

Output: $-2, -1, 0, 10, 11$

Practice Question 2: Scope Shadowing vs. Local Variable Conflict

Question:

What will be the output of the following C code?

```
#include <stdio.h>
enum Value { X = 5, Y = 10 };

int main() {
    int X = 20;
    {
        enum Value { X = 30, Y = 40 };
    }
}
```

```

        printf("%d ", X);
    }
    printf("%d", X);
    return 0;
}

```

Detailed Explanation:

1. Global declaration: `enum Value { X = 5, Y = 10 } .`
2. Inside `main()` :
 - `int X = 20;` is declared. This local variable **shadows** the global enum constant `X` .
3. Inside the inner block `{ ... }` :
 - `enum Value { X = 30, Y = 40 };` declares a new inner enumeration type.
 - In this inner block, the new enum constant `X` shadows the outer `int X = 20` .
 - Therefore, `printf("%d ", X);` prints `30` .
4. Exiting the inner block:
 - The inner scope expires.
 - `printf("%d", X);` references the local variable `X = 20` .
5. Combined output: `30 20` .

Output: `30 20`

Practice Question 3: Duplicate Evaluated Case Labels via Preprocessor and Enum

Question:

Examine the following code:

```

#include <stdio.h>
#define OFFSET 2

int main() {
    enum Steps {
        INIT = 0,
        STEP1 = 1,
        STEP2 = OFFSET,
        NEXT = 2
    };

    int s = 2;
    switch (s) {
        case INIT: printf("0"); break;
        case STEP1: printf("1"); break;

```

```

        case STEP2: printf("2"); break;
        case NEXT:  printf("3"); break;
        default:    printf("D");
    }
    return 0;
}

```

Does this code compile successfully? If not, what is the compilation error?

Detailed Explanation:

1. Compute the values of the enumerators:
 - `INIT = 0`
 - `STEP1 = 1`
 - `STEP2 = OFFSET = 2`
 - `NEXT = 2`
2. `STEP2` and `NEXT` both resolve to the integer constant `2`.
3. In the `switch` block:
 - `case STEP2:` translates to `case 2:`.
 - `case NEXT:` translates to `case 2:`.
4. C language rules strictly prohibit duplicate `case` labels within the same `switch` block.
5. The compiler will reject this code with: `error: duplicate case value '2'`.

Result: Compilation Error (Duplicate case value)

5. Quick Revision & Edge-Case Checklist

- ☐ Default first enumerator = `0`; unassigned subsequent enumerators = `previous + 1`.
 - ☐ Enumerator constants are **rvalues** of type `int`; `&CONST` is a compile error.
 - ☐ Multiple enumerator names can hold identical numeric values.
 - ☐ In standard C, assigning an out-of-range integer to an `enum` variable is allowed without casting.
 - ☐ In `switch-case`, two distinct enum constants that resolve to the same numeric value will produce a **duplicate case value compile error**.
 - ☐ In C, `const int` cannot be used as a `switch case` label, but `enum` constants **can** because they are true compile-time integer constants.
 - ☐ Enumerators share the ordinary identifier namespace; declaring a local variable with the same name in the same scope causes a redefinition error.
-